# QR AI Lab

## 使用手冊：各資料類型的訓練與推論

表格 · 影像 · 序列 · 光譜 · 文字

版本 0.2 · 2026-10

本手冊對應 backend v0.1 與 trainer v0.1；介面截圖為第一版畫面，實際版面可能微調。

# 目錄

1. 平台在做什麼、角色與共通流程
2. 建立任務（Job）：欄位說明與推論模式
3. 資料前處理：選擇步驟並即時預覽
4. 表格資料（TABULAR）
5. 影像資料（IMAGE）：分類、偵測、分割量測、異常、影像品質基準
6. 序列資料（SEQUENCE）
7. 光譜資料（SPECTRUM）
8. 文字資料（TEXT）
9. 看懂結果：指標、leaderboard、log
10. 管理員：角色、資源池、GPU、佇列、使用率歷史、模型庫
11. 常見問題
12. 附錄：API 速查

# 1. 平台在做什麼、角色與共通流程

QR AI Lab 是公司內部的 AI 模型實驗室：你提出專案申請，管理員核准並分配運算配額，之後在專案裡指定資料、選模型（或交給 AutoML）訓練，把訓練好的模型拿來推論。平台不會保存你的資料集：上傳檔與從你的 MinIO / URL 抓下來的副本只存在於任務執行期間，任務一結束就刪除；保存的只有訓練產物、log 與指標。

## 1.1 角色

| 角色 | 能做什麼 |
|---|---|
| 一般使用者（user） | 提出專案申請、在討論區發問；成為專案成員後可建立資料來源與任務 |
| 專案 owner / maintainer | 管理成員、封存專案、設定預設模型 |
| 專案 member | 建立資料來源、上傳、建立 / 取消任務、設定預設模型 |
| 專案 viewer | 只讀：看任務、log、下載產物 |
| 管理員（admin） | 日常營運：在專案的「申請」分頁審核、重排佇列、強制中斷 / 重新執行 / 刪除任何任務、關閉或刪除討論串；資源池、配額、模型庫只能看 |
| 系統管理員（superadmin，固定一位，由 SUPERADMIN_USERNAME 指定） | admin 的一切，加上：設定總資源池與工具池、在專案「設定」分頁調配額、刪除專案、上傳 / 匯入模型庫權重、把使用者設成 admin / user、強制釋放 GPU 顯存 |

登入方式由系統管理員設定：開發 / 過渡期用平台自己的帳號密碼；正式環境按「使用公司帳號登入（Keycloak）」走公司單一登入，第一次登入會自動建立帳號（Keycloak 只提供身分），角色由系統管理員在「使用者」頁指定。

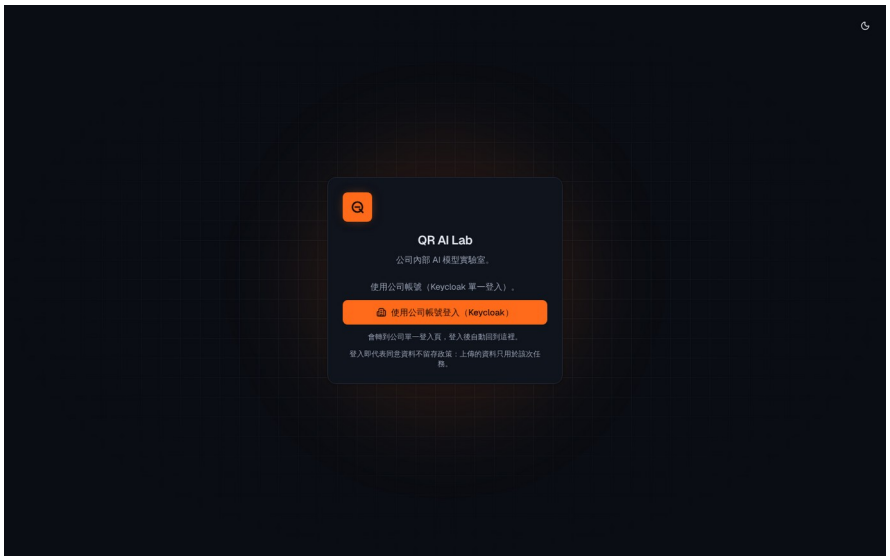



*圖 0　正式環境的登入頁（Keycloak）*

## 1.2 從申請到推論的流程

1. 登入（過渡期為本地帳號；之後接公司 Keycloak）。
2. 「專案申請」填題目、資料類型（TABULAR / IMAGE / SEQUENCE / TEXT / SPECTRUM）、任務類型（下拉只列出該資料類型有模型能做的：表格與光譜可做分類 / 迴歸 / 異常偵測，影像可做分類 / 偵測 / 分割 / 異常偵測，序列只有異常偵測，文字是分類 / 異常偵測）、需要的 GPU / CPU / 記憶體 / 儲存。

3. 管理員審核：核准時指定專案代號（slug）、配額與成員，系統自動建立專案。
4. 專案 →「資料來源」：新增你自己的 MinIO / URL（只存連線資訊，憑證加密），或在建立任務時一次性上傳檔案。
5. （可選）在上傳後選幾個資料前處理步驟，平台抽樣套用給你看前後對照。
6. 「新建任務」：選訓練，資料來源 + spec（例如目標欄位），自動模式（AutoML）或手動選模型，送出後進入全域佇列。
7. 任務頁看即時 log、進度、指標；完成後在「模型」頁看到產物，第一個模型自動成為預設模型。
8. 「新建任務」選推論：選模型、推論模式、輸入資料（直接貼 JSON、上傳或資料來源），完成後下載 predictions.csv。




*圖 1　專案申請表單（填好的例子）*

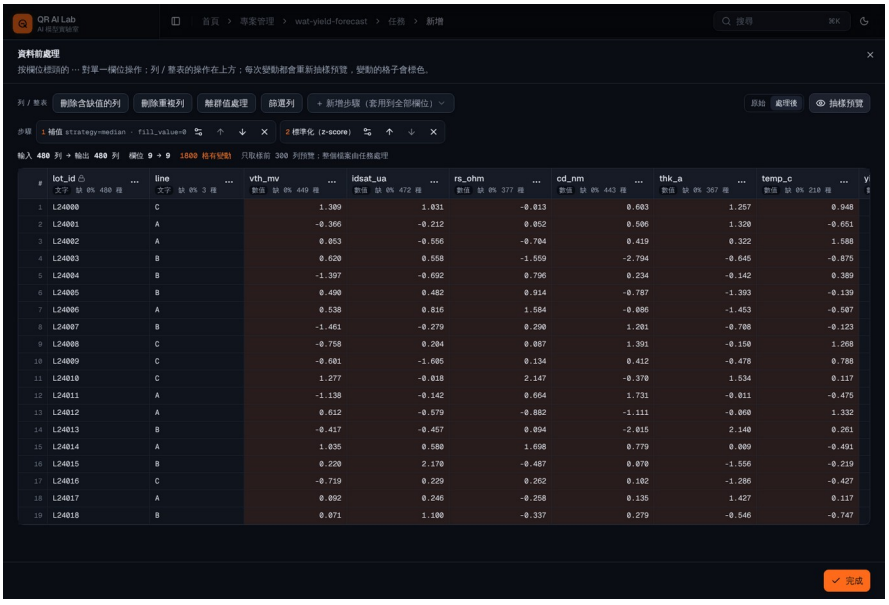



*圖 2　新建任務精靈：欄位與前處理（抽樣預覽處理前 / 後）*




*圖 3　任務頁：執行中的即時 log*

# 2. 建立任務（Job）：欄位說明與推論模式

訓練與推論都是「任務」，共用同一條全域佇列；管理員可以重排佇列或強制中斷。每個任務都有 log 檔與即時事件。

| 欄位 | 說明 |
|---|---|
| kind | train（訓練）或 inference（推論） |
| name | 任務名稱，方便辨識 |
| input.kind | upload（一次性上傳，任務結束即刪）、data_source（你的 MinIO / URL）、inline（推論用的小量 JSON） |
| input.spec | 資料的描述：target（目標欄）、features、categorical、text_column、label_column、window / stride、label、preprocess（前處理步驟）…依資料類型而定，見各章 |
| mode | 訓練：auto（AutoML，試多個候選模型並搜尋超參數）或 manual（指定 model_family + config） |
| model_family | 手動模式的模型家族，例如 tabular.xgboost、vision.yolo、spectrum.masked_autoencoder |
| config | 手動模式的超參數（如 epochs、max_depth）；推論用 config.mode 指定推論模式、config.k 等選項 |
| artifact_id | 推論用：要用哪個模型產物（預設模型會自動帶入） |
| requested_gpu / cpu / memory_mb | 資源請求，不能超過專案配額；需要 GPU 的家族至少 1 GPU |
| timeout_minutes | 逾時自動中止（預設 240 分鐘） |

## 2.1 推論模式（config.mode）

先想清楚推論時要做什麼，再回推要訓練哪個模型。不填 mode 就用該模型支援的第一個模式。

| 模式 | 白話 | 輸出 | 哪些模型 |
|---|---|---|---|
| predict | 給一筆資料回類別或數值 | prediction、confidence / 各類機率 | 表格、文字、DINO few-shot、YOLO 分類、光譜 MAE 迴歸頭 |
| score | 這筆資料正不正常 | anomaly_score、is_anomaly | Isolation Forest、Patch AE、光譜 MAE、PatchCore（附熱圖）、影像品質基準 |
| compare | 整批影像與黃金機台基準比對 | 每個指標 match / drift / fail 與總判定 | 影像品質基準 |
| embed | 資料 → 向量 | embedding、embeddings.npy | 所有深度學習家族、影像品質基準 |
| retrieve | 找最相似 / 最不相似、有多新 | most_similar、least_similar、novelty | Patch AE、光譜 MAE、DINO few-shot |
| detect / segment | 圖裡東西在哪、哪些像素 | 框、遮罩、標註圖 | YOLO |
| measure | 從遮罩算幾何量 | 面積、寬高、等效直徑（可換算 nm） | YOLO segmentation |

| 模式 | 白話 | 輸出 | 哪些模型 |
|---|---|---|---|
| extend | 不重訓加入新類別 | 新版本模型（自動設為預設） | DINO few-shot |

## 2.2 任務狀態

queued（排隊）→ scheduled（已分配資源）→ running（執行中）→ succeeded / failed / cancelled。排隊中會顯示等待原因（例如專案同時執行上限、資源池 GPU 不足）。

# 3. 資料前處理：選擇步驟並即時預覽

每種資料類型都有一組可選的前處理步驟。你在上傳後選步驟與參數，平台會從上傳檔抽樣（表格最多 5000 列統計、影像最多 8 張、曲線最多 50 條）套用並回傳前後對照；確認後把步驟放進任務的 input.spec.preprocess。訓練時這些步驟會「學」在訓練資料上（例如補值用的中位數、標準化的平均與標準差、對位的參考影像），並存進模型；推論時自動沿用同一組步驟與數值，不必再選。

> **注意**　偵測 / 分割任務不能用會改變座標的步驟（縮放、裁切、翻轉、旋轉、對位），因為標註框會失效；平台會直接拒絕。

## 3.1 可用步驟

| 資料類型 | 步驟（op id） |
|---|---|
| TABULAR | 補值 tabular.impute、刪除含缺值列 tabular.drop_missing、離群值截斷 / 刪除 tabular.clip_outliers、標準化 tabular.standardize、歸一化 tabular.minmax、對數 tabular.log、轉數值 tabular.to_numeric、篩選列 tabular.filter_rows、移除 / 只保留欄位、刪除重複列 |
| IMAGE | 縮放 image.resize、灰階、裁切 image.crop / center_crop、自動對比、直方圖均衡、Gamma、百分位拉伸 image.stretch、去雜訊 image.denoise、銳利化、反相、翻轉、旋轉、對位 image.align（相位相關求平移，參考影像存進模型） |
| SEQUENCE / SPECTRUM | 重取樣 sequence.resample、裁切範圍、補缺值、Savitzky–Golay 平滑、移動平均、微分 sequence.derivative、基線校正（多項式 / 滾動最小值）、去趨勢、SNV、每條歸一化、MSC sequence.msc、整體標準化、截斷數值 |
| TEXT | 轉小寫、清理空白、移除標點 / 網址 / 數字、截斷長度、正規表達式取代、刪除空白文字、刪除重複文字 |

## 3.2 操作

1. 上傳檔案後，在「前處理」區塊新增步驟，調整參數。
2. 按「預覽」：表格看每欄缺值數、平均、標準差、最小最大與前幾列；影像看縮圖；曲線看前後曲線。
3. 滿意後送出任務；任務 log 會印出 preprocess (fit): … 與列數變化。
4. 推論任務若使用這個模型，平台會拒絕再指定 preprocess（因為模型已內含），避免重複處理。

> **建議**　先用少量步驟。補值與標準化對表格模型（尤其線性、SVM）幫助最大；影像先做自動對比 / 百分位拉伸再談其他；光譜常用 SNV 或 MSC + Savitzky–Golay。

## 3.3 不用專案的資料前處理服務

左側選單「資料前處理」：不需要專案也能用。選資料類型 → 上傳檔案 → 加入步驟並抽樣預覽（即時，只取樣）→ 「建立前處理任務」。整個檔案的處理是公共工具池裡的任務（不佔用任何專案的配額，每人同時有數量上限）：和其他任務一樣排隊、有即時 log；完成後到「工具」裡的任務頁下載 zip（processed.csv / .npy / images/、pipeline.json、summary.json）。上傳檔在任務結束時刪除，結果 zip 下載一次就刪、沒下載的在管理員設定的時間（預設 24 小時）後刪。

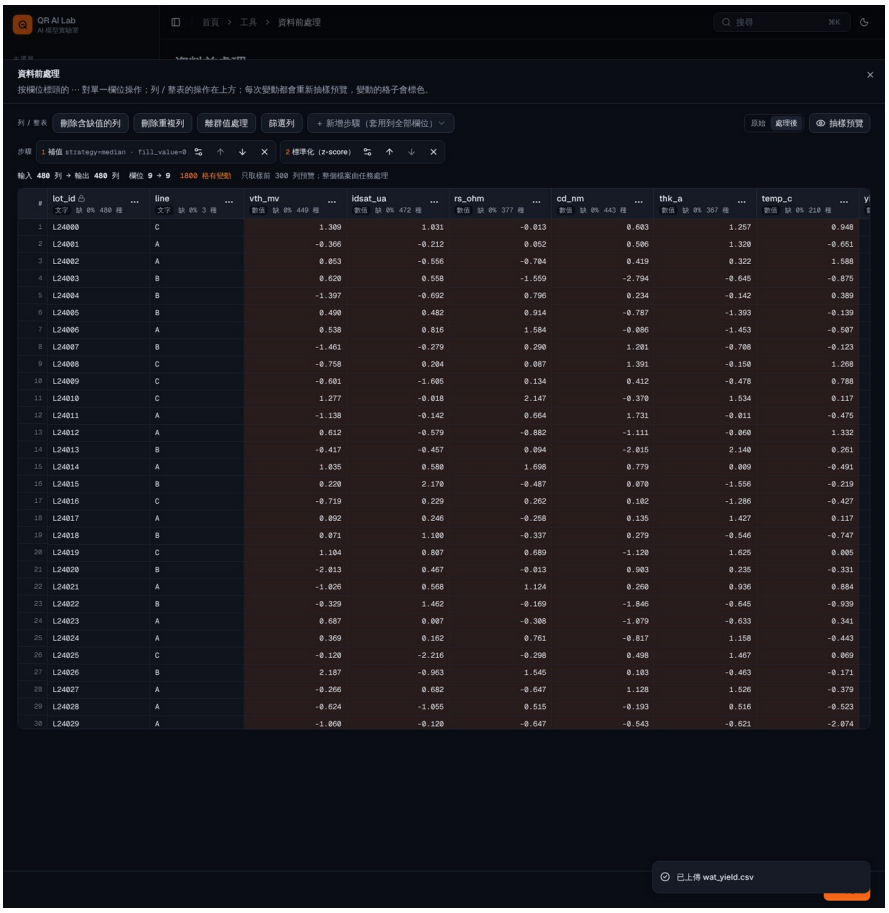



*圖 3a　資料前處理服務：上傳、步驟、抽樣預覽*

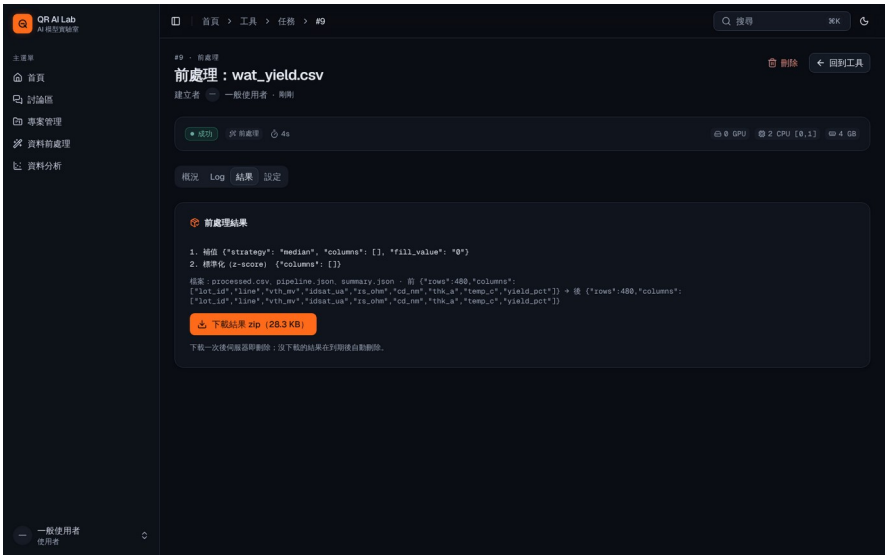



*圖 3b　前處理任務頁：log、步驟、一次性下載*

## 3.4 資料分析服務

左側選單「資料分析」：上傳資料、選分析類型，由任務算出報告。表格資料探勘：欄位概況、相關矩陣與高相關欄位對、VIF 共線性、指定目標欄位後每個欄位的解釋力（互資訊、相關性、隨機森林置換重要度）與疑似洩漏，最後給建議（補值、該丟哪些欄位、適合的模型）。影像品質分析：把不同機台 / 條件的影像放在 zip 的不同子資料夾，報告列出每個 IQA 指標在群組間的飄移比與 p 值、各群組中位數、embedding 的散佈圖與群組相似度、偏離群組的影像，以及該監控哪些指標。

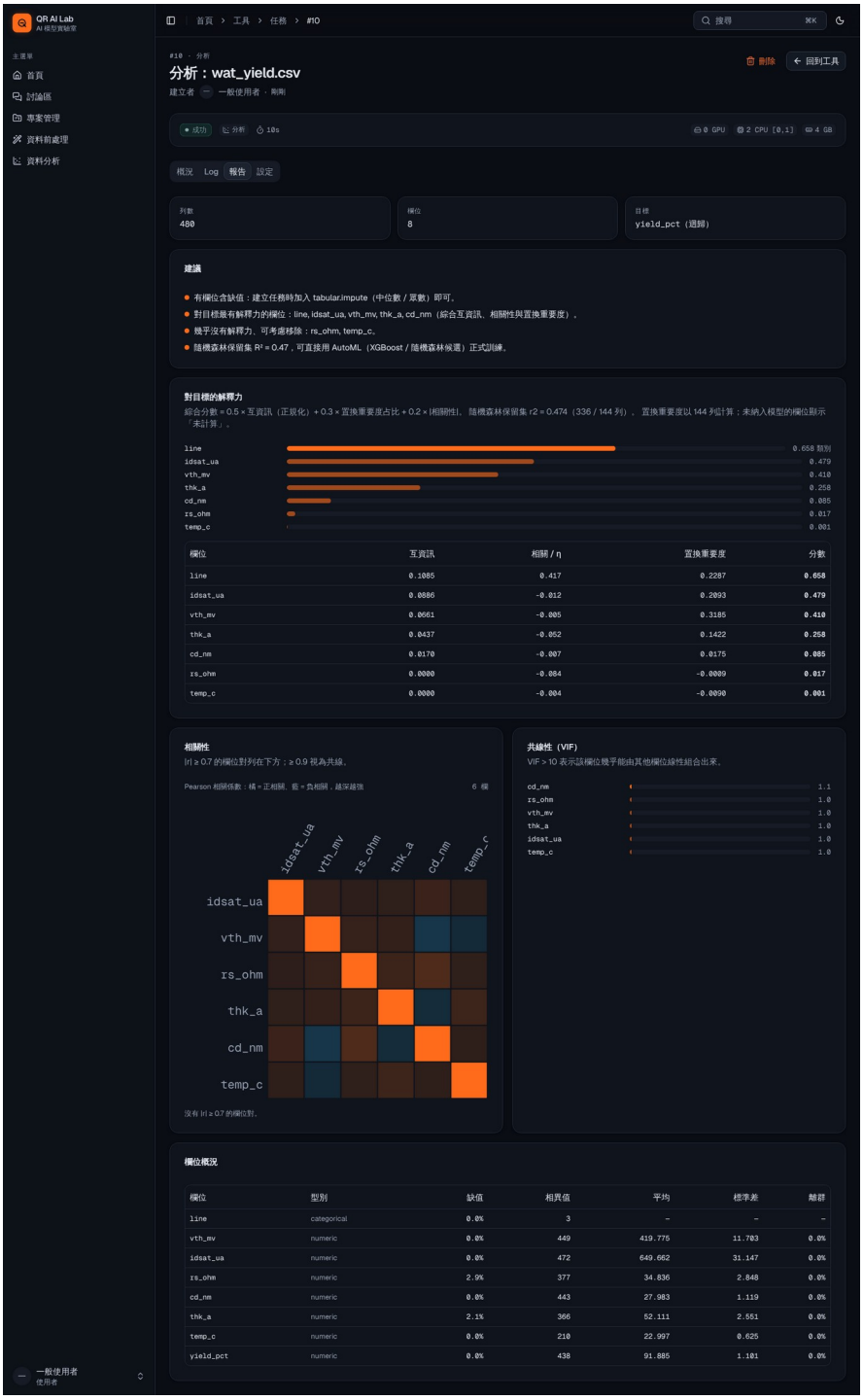



*圖 3c　表格資料探勘報告*




*圖 3d　影像品質分析報告（三台機台）*

# 4. 表格資料（TABULAR）

## 4.1 準備資料

- csv 或 parquet，第一列是欄名；每列一筆樣本、每欄一個特徵。
- spec.target：目標欄（分類用類別、迴歸用數值）；異常偵測不需要目標，但若有 0 / 1 或 normal / anomaly 標籤可填 target 供評估。
- 可選 spec.features（只用這些欄）、spec.categorical（類別欄；不填會自動偵測文字欄）、spec.drop_columns、spec.id_column（識別欄，不參與訓練，推論結果會帶回）。
- 缺值可以留空（平台會補），類別欄自動 one-hot。建議 ≥ 100 列；XGBoost 建議 ≥ 200 列。

## 4.2 模型

| 家族 | 適合 | 備註 |
|---|---|---|
| tabular.xgboost | 大多數分類 / 迴歸 | 表格首選；給特徵重要度 |
| tabular.random_forest | 小資料、不想調參 | 穩健；特徵重要度 |
| tabular.linear | 想解釋、當基準線 | 邏輯迴歸 / Ridge；秒級 |
| tabular.svm | 小資料、特徵多 | > 5 萬列會很慢 |
| tabular.isolation_forest | 只有正常資料找離群 | 任務類型 anomaly_detection，推論 score |

## 4.3 訓練（AutoML 範例：WAT 參數 → 良率）

1. 資料來源：上傳 wat.csv 或指定 MinIO 路徑。
2. 任務類型 regression，spec.target = yield_pct，可加 preprocess：impute + clip_outliers。
3. mode = auto，CPU 2、記憶體 4 GB、GPU 0。AutoML 會依序試 XGBoost、隨機森林、線性、SVM（> 5 萬列自動略過 SVM）。
4. 完成後看 leaderboard：每個 trial 的家族、參數、rmse；最佳模型自動匯出並登記，report/feature_importance.json 列出重要特徵。
5. 候選家族：XGBoost、CatBoost（另一個梯度提升樹，小資料與類別欄較穩）、TabPFN（預訓練好的表格 transformer，fit 只是把訓練列放進上下文、沒有超參數要調，幾十到幾千列常常直接最準；權重來自模型庫，> 1 萬列自動跳過）、隨機森林、線性、SVM。

## 4.4 推論

- inline：直接貼 rows，例如 [{"x1": 1.2, "x2": 0.3, "line": "A"}]；欄位缺的會補缺值，多的會忽略。
- upload / data_source：同格式的 csv（不需要目標欄）。
- 結果 predictions.csv：id、prediction、confidence（分類另有 proba_<類別>）；異常偵測：anomaly_score、is_anomaly。

```
{"kind":"train","name":"wat-yield","task_type":"regression","mode":"auto",
 "input":{"kind":"upload","upload_id":12,"spec":{"target":"yield_pct",
   "preprocess":[{"op":"tabular.impute"},{"op":"tabular.clip_outliers","params":
{"action":"clip"}}]}},
 "requested_gpu":0,"requested_cpu":2,"requested_memory_mb":4096}
```

# 5. 影像資料（IMAGE）

## 5.1 準備資料

| 任務 | 格式 | 備註 |
|---|---|---|
| classification | zip：<類別>/*.jpg（可再分 train/、val/） | few-shot 每類 ≥ 3 張；YOLO 分類每類 ≥ 50 張 |
| detection / segmentation | zip：YOLO 格式<br>images/、labels/、data.yaml | 分割用多邊形標註 |
| anomaly_detection | zip：good/（或 normal/、ok/）為正常；其他資料夾視為異常（只用於評估） | PatchCore 與影像品質基準都用這個格式 |

## 5.2 分類：bin code 會一直增加 → DINO few-shot

- 家族 vision.dino_fewshot：凍結 DINOv2 或 DINOv3（config.backbone = dinov2-small / dinov2-base / dinov3-vits16 / dinov3-vitb16，權重來自模型庫）取特徵，每類存一個原型；每類幾張範例就能用。需要 GPU 1。DINOv3 的 patch 較細、SEM / TEM 紋理通常較穩，但權重是 gated（管理員要先在 Hugging Face 接受授權）。
- 推論 predict：回最近的類別與相似度；config.min_similarity 可讓低相似度的回 unknown。retrieve：找最相似 / 最不相似的範例影像，表單可選方向（只找最相似、只找最不相似、兩者都列）與每筆回幾個；最不相似用來找離群或挑下一批要標註的。
- 看模型在看哪裡：predict / retrieve 每張影像都會多一張「模型關注的區塊」疊圖；在結果列點開樣本，疊圖會以可調透明度覆蓋在你瀏覽器裡的原圖上（紅 = 影響判斷最多的位置；只覆蓋模型實際看到的中央區域）。紅色落在背景或資訊列時，先用前處理 image.crop 切掉再訓練。
- 新增 bin code 不用重訓：建立推論任務，config.mode = extend，輸入新類別的資料夾 zip，平台產生新版本模型並設為預設。

## 5.3 分類：類別固定、每類幾十張以上 → DINO 微調

- 家族 vision.dino_finetune：真的訓練 DINOv2 / DINOv3 做分類。config.method = linear 只訓練分類頭（最快）；lora 在注意力層加低秩轉接器一起訓練（SEM / TEM 這類和自然影像差很多的資料用這個，產物只有幾 MB）；full 全參數微調（每類上百張、顯示卡記憶體夠時）。
- 其他參數：epochs、lr、rank（LoRA 秩）、batch、augment（訓練時的隨機增強，自己挑：hflip、vflip、rotate90、crop、brightness、contrast；方向有意義的影像不要加翻轉 / 旋轉）；自動模式會在 linear / lora 之間搜尋。推論 predict 回 prediction、confidence、top3（config.min_confidence 可讓低信心的回 unknown），並附 Grad-CAM 疊圖（分類頭對每個區塊的貢獻）；retrieve 找最像 / 最不像的訓練影像（可選方向）；embed 輸出特徵向量。
- few-shot 與微調怎麼選：類別會一直增加、每類只有幾張 → few-shot；類別固定、每類幾十張以上、要更準 → 微調。

## 5.4 分類：類別固定、資料多 → YOLO 分類

- 家族 vision.yolo，task classification；config.model = yolov8n / s / m、imgsz、epochs；pretrained = true 從模型庫的 yolov8s-cls.pt 微調，false 從頭訓練。augment 是多選

（hflip、vflip、rotate90、crop、brightness；預設 hflip + crop + brightness），偵測 / 分割任務也一樣。

- 指標 accuracy、f1_macro；推論 predict 回 prediction、confidence、top5。

## 5.5 偵測與分割量測

- YOLO 偵測：推論 detect，每個物件一列（class、confidence、x1 y1 x2 y2）；output/images/ 有標註圖。
- YOLO 分割：推論 segment 或 measure；measure 多給 area_px、width_px、height_px、equiv_diameter_px，config.pixel_size（每像素的實際長度，例如 2.5 nm）會再給 *_phys 欄。

## 5.6 只有良品影像 → PatchCore 異常偵測

- 家族 vision.dino_patchcore（GPU）：正常影像的 patch 特徵建記憶庫；推論 score 回 anomaly_score、is_anomaly、hot_patch_row / col，並輸出 heatmaps/*.png。

## 5.7 影像品質基準與機台比對（tool matching）

場景：用黃金機台的一批影像建立「影像品質」基準與浮動範圍，其他機台調機後拍同類樣品，檢視每個品質指標是否落在基準範圍內。不需要標註、不需要 GPU、不需要預訓練權重。

- 家族 vision.image_quality，task anomaly_detection。訓練資料：黃金機台影像放 good/（或整包都是正常）。
- 17 個無參考指標：清晰度（Laplacian 變異、Tenengrad、Brenner）、雜訊標準差、訊噪比、對比（RMS、Michelson）、亮度（平均、P1、P99）、過暗 / 過亮飽和比例、亮度熵、高頻能量比、頻譜斜率、方向性（梯度 x/y、頻譜水平 / 垂直，可反映像散或掃描方向問題）。
- 基準 = 每個指標的穩健中位數與尺度（MAD），浮動範圍 = 中位數 ± z_threshold × 尺度（預設 3）；另以 Mahalanobis 距離做整體判定，門檻取黃金影像距離的 P99。產物 report/baseline_ranges.csv 列出每個指標的範圍。
- 推論 compare：把調機後機台的一批影像丟進來，每個指標給 golden_median、golden_low / high、test_median、effect（偏移多少個尺度）、KS 檢定 p 值、in_range_ratio 與 verdict（match / drift / fail）；總判定 pass / drift / fail，並列出 failed_metrics、drifted_metrics。output/comparison.csv 就是調機報告。
- 推論 score：單張影像的指標、z 分數、距離與 is_anomaly，用來做日常監控。
- 該調什麼：17 個指標分成五個面向（對焦 / 清晰度、雜訊、亮度、對比、方向性 / 像散），結果頁的「建議調整」卡片依偏離的面向與方向給出動作（例如比基準模糊 → 重新對焦、校正像散、確認工作距離；方向性不同 → stigmator X / Y、掃描方向）；score 的每一列也有 advice 欄。
- 偏離在哪裡：每張影像有一張「偏離基準最多的區塊」疊圖（偏離最多的指標依 8 × 8 方格著色），點開樣本就疊在原圖上：整張一致 = 機台設定，集中在某一角 = 樣品不平、帶電或掃描問題。
- 同一樣品在兩台機台逐張比對（全參考）：zip 放 golden/（參考機台）與 target/ 兩個資料夾、同檔名，compare 會自動逐張對位後算 PSNR、SSIM、MS-SSIM、GMSD，結果頁多一張「與參考影像逐張比對」卡片（pass / drift / fail），output/paired.csv 逐張；疊圖改成差異最大的區塊。config.paired = false 可關閉。

- 選項：config.z_threshold、config.effect_warn（預設 1）、config.effect_fail（預設 2）、config.alpha（KS 顯著水準，預設 0.01）、config.ssim_warn（0.9）/ ssim_fail（0.7）、config.maps（疊圖，預設開）。

> **建議**　同一基準只適用於同類樣品與同放大倍率；不同製程層或倍率請分別建基準。影像上若有資訊列（SEM 底部文字），先用前處理 image.crop 切掉再建基準。

## 6. 序列資料（SEQUENCE）

- 格式：csv 長表 series_id,t,feature1,feature2…（spec.window、spec.stride 切視窗；spec.series_id、spec.time 指定欄名）或 npy（N × T × C）。
- 家族 sequence.patch_autoencoder：patch 嵌入 + 位置編碼 + 整段視窗壓到一個 latent，「峰值移位」也能抓到；CPU 可跑，GPU 更快。
- 訓練：task anomaly_detection；有 spec.label（normal / anomaly）時只用正常視窗訓練，異常視窗進驗證集算 auroc。
- 推論：score（異常分數、門檻可用 config.threshold 覆寫）、retrieve（找歷史最像 / 最不像的 run，config.k；config.reference = self 可讓輸入互相比）、embed（向量給 BI 分群）。
- 前處理常用：sequence.fill_nan、sequence.detrend、sequence.standardize。

## 7. 光譜資料（SPECTRUM）

- 格式：spectrum_csv 寬表（每列一條光譜，欄 = 波長）或 npy（N × T）；可選 spec.label（異常標籤、迴歸目標或類別）。推論時長度不同的光譜會自動重取樣到訓練長度。
- 有量測值或類別先用 spectrum.pls：PLS 迴歸（光譜 → CD / 膜厚 / 濃度）與 PLS-DA 分類（PLS 分數 + 邏輯迴歸）；幾十筆就能用、秒級、不用 GPU；config.n_components（自動模式搜 2 … 30）；產物 report/vip.csv 列出每個波長的 VIP（> 1 = 真的在用）。
- 家族 spectrum.masked_autoencoder：把光譜切 patch、隨機遮蔽再重建，學到的 embedding 當光譜鑑別器；候補 sequence.patch_autoencoder。
- 光譜鑑別器（OCD / 橢圓偏光）：訓練 anomaly_detection → 推論 retrieve 回每條光譜最像 / 最不像的黃金樣本 id、max_similarity、novelty，summary.most_novel 直接列出最可疑的；score 看異常；定期跑 retrieve 追蹤 max_similarity 中位數可看漂移。
- 光譜 → CD / 膜厚（資料多時）：task regression + spec.label 為量測值；自動模式先試 PLS，≥ 500 筆再加入 MAE（embedding 接 Ridge 迴歸頭；推論 predict 多回 reconstruction_error，誤差大代表不在訓練分布內，預測值別信）。
- 前處理常用：sequence.savgol、sequence.baseline_rolling_min（螢光背景）、sequence.snv 或 sequence.msc、sequence.crop（去兩端雜訊）。

## 8. 文字資料（TEXT）

- 格式：jsonl 或 csv，spec.text_column（預設 text）、spec.label_column（預設 label）。
- 分類：AutoML 先試 text.qwen_embedding（凍結的 Qwen3-Embedding 向量 + 邏輯迴歸，幾十筆就能用、沒有 GPU 也跑得動）與 TF-IDF + 邏輯迴歸（tabular.linear）；≥ 300 筆再加入 text.bert_classifier（微調 bert-base-chinese / xlm-roberta-base，需要 GPU）。
- 推論 predict：prediction、confidence、各類機率；embed：向量。
- 找相似工單、新型問題：text.qwen_embedding 的 retrieve（可選只找最相似 / 只找最不相似 / 兩者，每筆回幾個；config.reference = self 讓一批工單互比找重複）；task anomaly_detection 時 score 回 novelty 分數（和歷史紀錄都不像 = 新型問題，門檻 = 歷史文字的 P99）。模型只存向量與 id，不存原文。
- 前處理常用：text.strip_whitespace、text.remove_urls、text.dedupe。

## 9. 看懂結果：指標、leaderboard、log

| 任務 | 主要指標 | 怎麼讀 |
|---|---|---|
| 分類 | f1_macro、accuracy、auroc | f1_macro > 0.8 通常可用；不平衡資料看 f1_macro 不看 accuracy |
| 迴歸 | rmse、mae、r2 | rmse 要跟目標範圍比；r2 越接近 1 越好 |
| 偵測 / 分割 | map50_95、map50 / mask_map50_95 | map50 > 0.5 可試用 |
| 異常偵測 | auroc、f1、threshold、flagged_ratio | auroc > 0.9 才可靠；沒標籤時只能看分數分布 |
| 影像品質比對 | verdict、match_ratio、effect、p_value | effect 絕對值 < 1 且 p > 0.01 為 match；≥ 2 為 fail |

- AutoML 的 leaderboard 列出每個 trial 的家族、參數、指標與耗時；最佳 trial 的參數可以直接拿去建手動任務微調。
- log 每行有時間與等級：[SYSTEM] 是排程器（分配資源、啟動、逾時、中斷），[INFO] / [WARN] / [ERROR] 來自訓練程式；每個 epoch / trial 一行。
- 結果不好：先檢查資料（標籤錯、類別極端不平衡、圖片太小、光譜長度不一致），再增加資料，最後才調參。

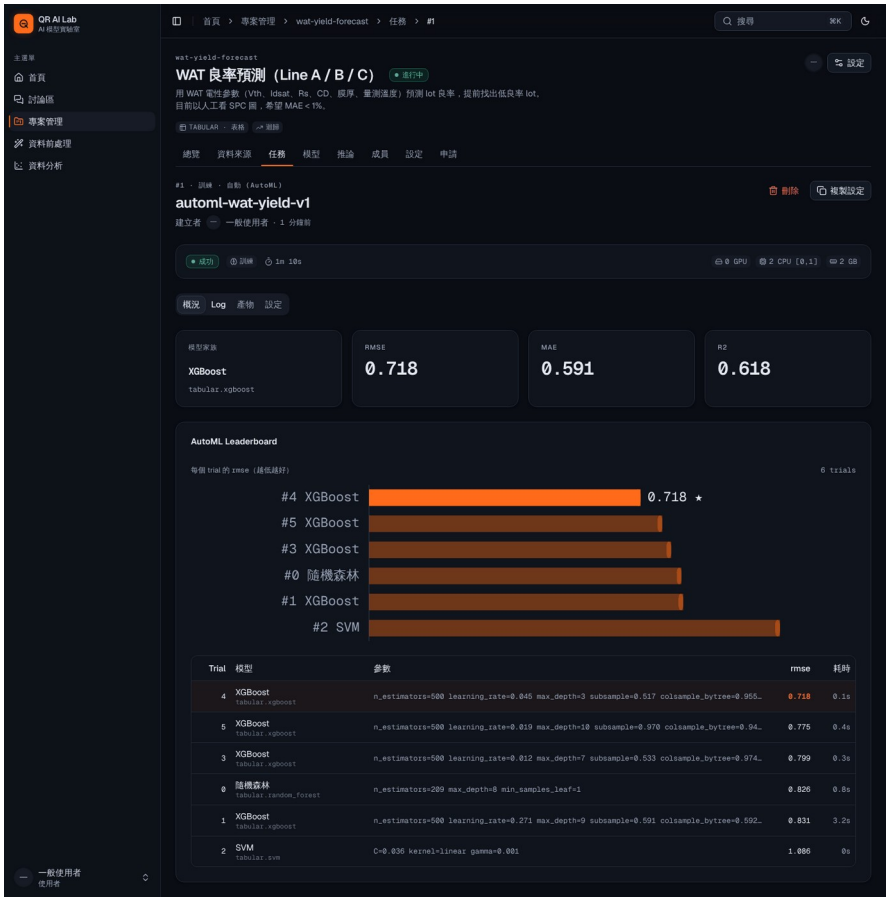



*圖 4　訓練完成：最佳模型、AutoML leaderboard*

## 10. 管理員：角色、資源池、GPU、佇列、使用率歷史、模型庫

- 角色（superadmin）：「使用者」頁把人設成 admin / user 或停用；superadmin 只有一位，由伺服器設定 SUPERADMIN_USERNAME 指定，介面上不能指派。Keycloak 只提供身分，不決定角色。
- 總資源池（superadmin）：所有專案合計可用的 GPU 索引、CPU 核心、記憶體與同時執行上限；排程器以此為硬上限，並允許 backfill。「依顯存共用 GPU」打開時，任務建立時填了「每張 GPU 需要的顯存」就能和其他任務共用同一張卡。
- GPU 卡片與重置（superadmin）：每張卡顯示驅動程式讀到的顯存、排程器保留量、放在上面的任務；「重置」可只中斷本系統的任務、連其他程序一起 kill，或執行伺服器設定的 nvidia-smi --gpu-reset。
- 佇列：看排隊與執行中的任務，置頂 / 上下移動重排；每次重排都寫稽核紀錄。
- 強制中斷 / 重新執行 / 刪除（admin 以上）：任務頁與佇列頁都有。重新執行會用相同輸入與設定建立新任務（一次性上傳的任務除外，因為資料已經刪掉）。
- 使用率歷史：節點實際用量、配置給任務的量、GPU 使用率與顯存、各專案占用；可切 1 小時到 30 天，分母可選專案配額 / 資源池 / 系統總量。
- 模型庫（superadmin 才能改）：列出目錄需要的預訓練權重與是否存在，三種帶進來的方式：瀏覽器上傳、伺服器路徑、內網 URL；訓練程序完全離線。權重來源以 Hugging Face Hub 為主（YOLO 偵測版：Ultralytics/YOLOv8；DINOv2 / v3：facebook；BERT；Qwen3-Embedding；TabPFN：Prior-Labs；YOLO 的分割 / 分類權重 Hugging Face 沒有，從 ultralytics 的 GitHub release 抓），有網路的機器用 scripts/fetch_model_zoo.py 一次抓齊（DINOv3 為 gated 模型，要先接受授權並設定 HF_TOKEN，或用 --skip 略過）。
- 專案申請審核（admin 以上）：專案列表以「申請中 / 審核中」篩選就是待審清單；在專案的「申請」分頁開始審核、核准（設定 GPU / CPU / 記憶體 / 儲存配額、同時執行上限與成員）或退回。工具池（superadmin）：所有人前處理 / 分析任務共用的配額與每人同時上限。
- 刪專案（superadmin）：專案設定頁最下方，要輸入 slug 確認；有未結束任務時拒絕。刪討論串（admin 以上）：討論串頁的「刪除」。

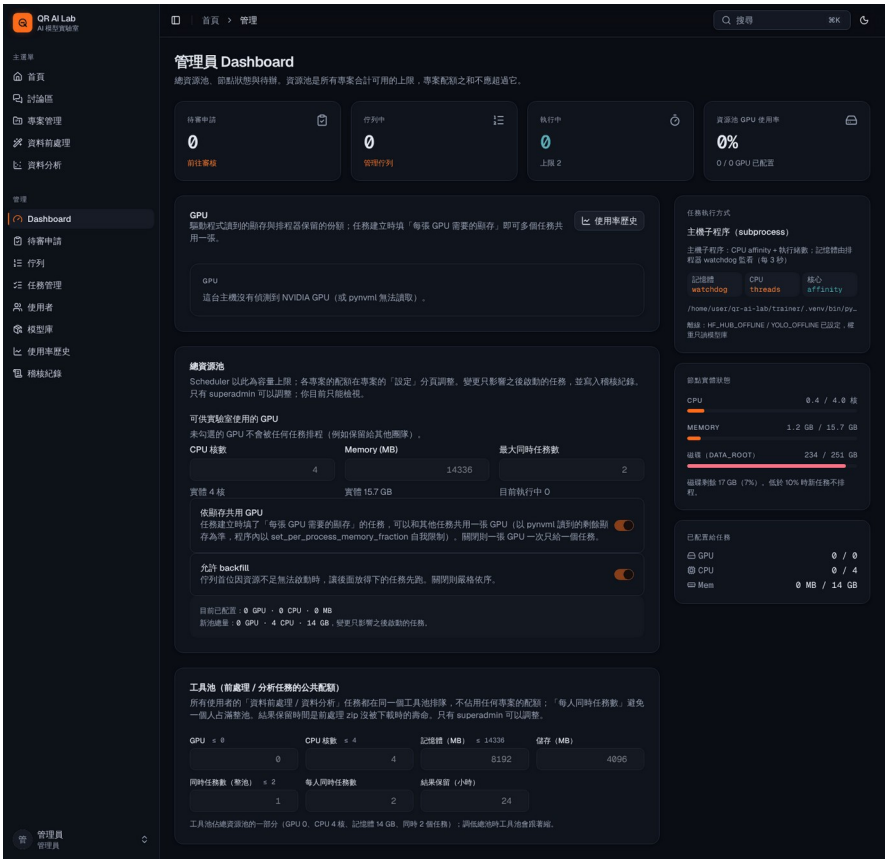



*圖 5　管理員儀表板*




*圖 6　佇列與執行中任務*




*圖 7　使用者與系統角色*




*圖 8　任務頁的管理員操作（強制中斷 / 重新執行 / 刪除）*

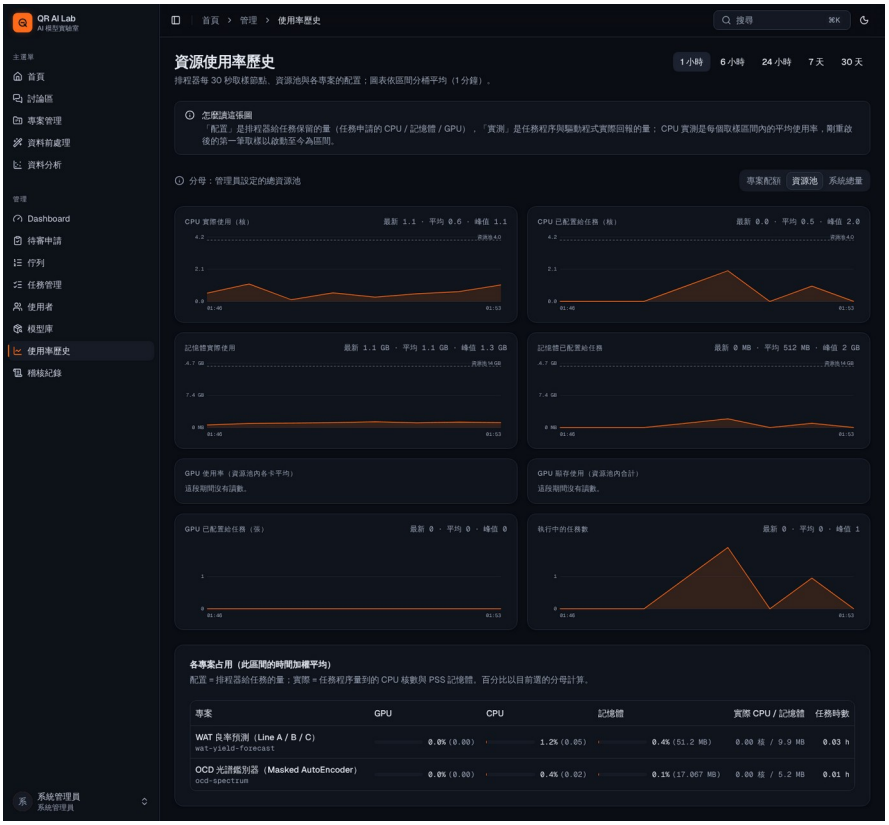

QR AI Lab
AI 模型實驗室
主選單
首頁
討論區
專案管理
資料前處理
資料分析
管理
Dashboard
待審申請
佇列
任務管理
使用者
模型庫
使用率歷史
稽核紀錄
系統管理員
系統管理員
首頁 › 管理 › 使用率歷史
搜尋 ⌘K
資源使用率歷史
排程器每 30 秒取樣節點。資源池與各專案的配置；圖表依區間分桶平均（1 分鐘）。
1 小時
6 小時
24 小時
7 天
30 天
怎麼讀這張圖
「配置」是排程器給任務保留的量（任務申請的 CPU / 記憶體 / GPU），「實測」是任務程序與驅動程式實際回報的量；CPU 實測是每個取樣區間內的平均使用率，剛重啟後的第一筆取樣以啟動至今為區間。
分母：管理員設定的總資源池
專案配額
資源池
系統總量
CPU 實際使用（核）
最新 1.1 · 平均 0.6 · 峰值 1.1
CPU 已配置給任務（核）
最新 0.0 · 平均 0.5 · 峰值 2.0
記憶體實際使用
最新 1.1 GB · 平均 1.1 GB · 峰值 1.3 GB
記憶體已配置給任務
最新 0 MB · 平均 512 MB · 峰值 2 GB
GPU 使用率（資源池內各卡平均）
這段期間沒有讀數。
GPU 顯存使用（資源池內合計）
這段期間沒有讀數。
GPU 已配置給任務（張）
最新 0 · 平均 0 · 峰值 0
執行中的任務數
最新 0 · 平均 0 · 峰值 1
各專案占用（此區間的時間加權平均）
配置 = 排程器給任務的量；實際 = 任務程序量到的 CPU 核數與 PSS 記憶體。百分比以目前選的分母計算。
專案
GPU
CPU
記憶體
實際 CPU / 記憶體
任務時數
WAT 良率預測（Line A / B / C）
wat-yield-forecast
0.0% (0.00)
1.2% (0.05)
0.4% (51.2 MB)
0.00 核 / 9.9 MB
0.03 h
OCD 光譜鑑別器（Masked AutoEncoder）
ocd-spectrum
0.0% (0.00)
0.4% (0.02)
0.1% (17.067 MB)
0.00 核 / 5.2 MB
0.01 h

*圖 9　資源使用率歷史*

# 11. 常見問題

| 狀況 | 原因與處理 |
|---|---|
| 任務一直 queued | 看等待原因：專案同時執行上限、GPU 配額、資源池不足；請管理員調整或等待 |
| GPU_REQUIRED | 選的家族需要 GPU（YOLO、DINO、BERT），申請時 requested_gpu ≥ 1；或改用 CPU 家族 |
| 模型庫缺少權重 | log 會說缺哪個檔與預期路徑；請管理員執行 fetch_model_zoo.py 並放到 MODEL_ZOO_ROOT |
| STORAGE_QUOTA_EXCEEDED | 專案儲存已滿；刪除舊產物或請管理員調高配額 |
| PREPROCESS_INVALID | 前處理步驟不適用於此資料類型 / 任務，或欄位不存在；用預覽確認 |
| 推論欄位對不上 | 表格推論會補缺的欄位並忽略多的欄位；確認欄名大小寫一致 |
| 影像品質 compare 全部 fail | 確認放大倍率、樣品種類與基準一致；先用前處理裁掉資訊列；基準影像至少 30 張 |
| 任務 failed：timeout | 調高 timeout_minutes 或縮小資料 / epochs |

# 12. 附錄：API 速查

前綴 /api/v1，所有請求帶 Authorization: Bearer <token>。

```
POST /auth/login {username, password}                    登入
GET  /catalog                                            資料類型、任務、模型家族、AutoML 規則
GET  /catalog/preprocess?data_type=TABULAR               可用前處理步驟與參數
POST /projects                                           專案申請（建立 pending 專案）；/{slug}/approve 管理員核准
POST /projects/{slug}/data-sources                       新增 MinIO / URL 資料來源；/{id}/check 連線檢查
POST /projects/{slug}/uploads  (multipart)               一次性上傳
POST /projects/{slug}/uploads/{id}/preview {steps,sample,spec}   前處理抽樣預覽
POST /projects/{slug}/jobs                               建立訓練 / 推論任務（202）
GET  /jobs/{id} | /logs | /events (SSE) | /artifacts     任務狀態、log、即時事件、產物
POST /jobs/{id}/cancel                                   取消
GET  /artifacts/{id}/download                            下載產物（目錄會打包成 zip）
GET  /admin/queue | PUT /admin/queue/order | POST /admin/jobs/{id}/stop   管理員
GET  /admin/model-zoo                                    模型庫狀態
```